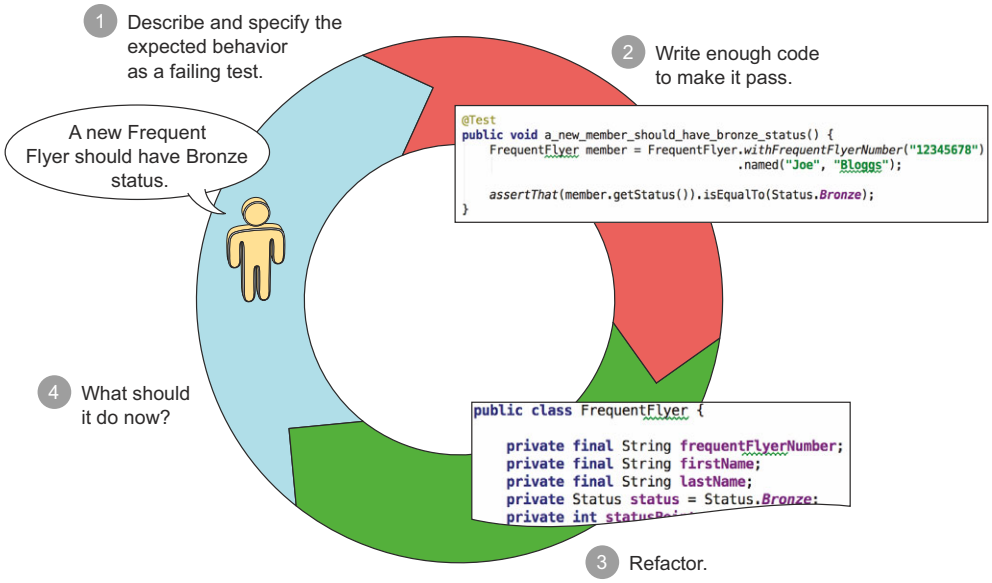


Figure 10.4 Use an incremental approach to progressively describe the behavior of a class.

To flesh out the application code required to implement the high-level step definitions, you write low-level specifications, expressed as unit tests, to progressively describe what your class should do. This approach of using unit tests to drive low-level design is a core TDD practice. The overall process is illustrated in figure 10.4, and can be summarized in the following steps:

1. Describe the behavior you need for a small piece of functionality in the form of a failing test.
2. Write just enough code to make this test pass.
3. Tidy up the code you just wrote, if required, to simplify it, remove duplication, and improve clarity.
4. Ask yourself "What else should the class do?" or "What input could I provide that should produce a different outcome?" and repeat the process.

When your acceptance criteria pass, you're done!

For example, the step definition for registering a new Frequent Flyer member discussed previously looks like this:

```
FrequentFlyer member;

@When("^(?:s?)he registers on the Frequent Flyer program$")
public void registers_on_the_Frequent_Flyer_program() throws Throwable {
    member = FrequentFlyer.withFrequentFlyerNumber("123456789")
                          .named(firstName, lastName);
}
```

Application code illustrating how to create a new Frequent Flyer member (1)